



Introduction to Pandas :-

What is Pandas?

Pandas is an open-source Python library used for data manipulation, analysis, and cleaning. It provides powerful and flexible data structures that make working with structured data fast and easy.

Pandas is built on top of the NumPy library and is widely used in Data Science, Machine Learning, Data Analysis, Artificial Intelligence, and Business Intelligence.

History of Pandas :-

Pandas was created by **Wes McKinney** in **2008** while working at **AQR Capital Management**. He needed a powerful and flexible tool to analyze financial data efficiently.

Before Pandas, Python mainly relied on NumPy for numerical computing. While NumPy was excellent for numerical arrays, it lacked convenient tools for handling tabular data, missing values, and labeled data.

To solve these limitations, Wes McKinney developed Pandas. The library introduced two powerful data structures:

- **Series** – A one-dimensional labeled array.
- **DataFrame** – A two-dimensional labeled table with rows and columns.

The name "**Pandas**" is derived from **Panel Data**, an econometrics term referring to multidimensional structured datasets.

Why Pandas was Developed :-

Before Pandas, Python mainly relied on NumPy for handling numerical data. Although NumPy is fast and efficient, it was not designed to work with structured tabular data such as spreadsheets or databases.

Pandas was developed to solve these limitations by providing easy-to-use tools for data manipulation, cleaning, analysis, and handling missing values.

Problems with NumPy

- Difficult to work with rows and columns.
- No labeled rows or columns.
- Handling missing values was not convenient.
- Limited support for reading and writing files like CSV and Excel.
- Complex syntax for data analysis tasks.

How Pandas Solves These Problems

- Provides labeled data structures (Series and DataFrame).
- Makes data cleaning simple.
- Handles missing values efficiently.
- Supports reading and writing multiple file formats.

Pandas vs NumPy

Feature	NumPy	Pandas
Data Type	Numerical data	Structured & tabular data
Speed	Fast	Fast
Labels	NO	YES
Missing Value Handling	Limited	YES
CSV/Excel Support	Limited	YES
Data Analysis	Basic	Excellent

Features of Pandas :-

Key Features of Pandas

- Provides powerful data structures (Series and DataFrame).
- Fast and efficient data manipulation.
- Easy handling of missing values.
- Supports labeled rows and columns.
- Reads and writes multiple file formats (CSV, Excel, JSON, SQL, etc.).
- Powerful indexing and selection.
- Easy filtering, sorting, and grouping.
- Built-in statistical functions.

Application of Pandas :-

Pandas is widely used in various industries because it makes working with structured data simple and efficient.

Major Applications of Pandas

- Data Analysis
- Machine Learning
- Data Science
- Financial Analysis
- Banking and Insurance
- Business Intelligence
- Data Cleaning and Preprocessing
- Web Data Analysis
- Healthcare Analytics
- Research and Scientific Computing
- Sales and Marketing Analysis
- Stock Market Analysis
- Database Management
- Report Generation

✓ Installing and Importing Pandas :-

Before using Pandas, it must be installed on your system. If you are using **Google Colab**, Pandas is already installed by default.

Install Pandas

Syntax :-

```
pip install pandas
```

Upgrade Pandas

Syntax :-

```
pip install --upgrade pandas
```

Import Pandas

Syntax :-

```
import pandas as pd
```

Using **pd** as an alias makes the code shorter and is the standard convention used by Python developers.

```
import pandas as pd

print("Pandas Version:", pd.__version__)
print("Pandas has been imported successfully!")
```

```
Pandas Version: 2.2.2
Pandas has been imported successfully!
```

✓ Pandas Data Structures (Series and DataFrame) :-

Pandas Data Structures -

Pandas provides two powerful data structures for storing and working with data.

1. Series

A **Series** is a one-dimensional labeled array that can store data of any type, such as integers, strings, or floating-point numbers. Each value in a Series has an associated index.

Example

Index	Value
0	10

Index	Value
1	20
2	30
3	40

2. DataFrame

A **DataFrame** is a two-dimensional labeled data structure with rows and columns, similar to an Excel spreadsheet or a database table. Each column can store a different type of data.

Example

Name	Age	City
Alice	22	Mumbai
Bob	25	Pune
Charlie	24	Delhi

Series		Series		DataFrame	
apples		oranges		apples	oranges
0	3	0	0	0	3
1	2	1	3	1	2
2	0	2	7	2	0
3	1	3	2	3	1

✓ Creating a Series :-

A **Series** is a one-dimensional labeled array in Pandas. It can store different types of data, such as integers, floats, strings, and even Python objects.

A Series consists of two parts:

- **Index** – Identifies each element.
- **Value** – The actual data stored.

Syntax :-

```
pd.Series(data, index)
```

Parameters :-

- **data** : The values to store in the Series.
- **index** (*optional*) : Labels for each value. If not provided, Pandas automatically assigns numeric indexes starting from 0.

Create a Series from a List :-

```
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50])

print(s)
```

0	10
1	20
2	30
3	40
4	50

dtype: int64

Create a Series with Custom Index :-

```
import pandas as pd

s = pd.Series([10, 20, 30], index=["A", "B", "C"])

print(s)
```

A	10
B	20
C	30

dtype: int64

Create a Series from a Dictionary :-

```
import pandas as pd

student = {
    "Name": "Alice",
    "Age": 22,
    "City": "Mumbai"
}

s = pd.Series(student)

print(s)
```

```
Name      Alice
Age        22
City      Mumbai
dtype: object
```

Accessing Elements in a Series :-

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])

print(s[0])
print(s[2])
```

```
10
30
```

✓ Creating a DataFrame :-

A **DataFrame** is a two-dimensional labeled data structure in Pandas. It stores data in rows and columns, similar to an Excel spreadsheet or a SQL table.

A DataFrame can contain different types of data in different columns.

Syntax :-

```
pd.DataFrame(data)
```

Parameters :-

bold text

- **data** : The data used to create the DataFrame. It can be a dictionary, list, list of dictionaries, NumPy array, or another DataFrame.

Create a DataFrame from a Dictionary :-

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 25, 24],
    "City": ["Mumbai", "Pune", "Delhi"]
}

df = pd.DataFrame(data)
```

```
print(df)
```

	Name	Age	City
0	Alice	22	Mumbai
1	Bob	25	Pune
2	Charlie	24	Delhi

Create a DataFrame from a List of Dictionaries :-

```
import pandas as pd

data = [
    {"Name": "Alice", "Age": 22, "City": "Mumbai"},
    {"Name": "Bob", "Age": 25, "City": "Pune"},
    {"Name": "Charlie", "Age": 24, "City": "Delhi"}
]

df = pd.DataFrame(data)

print(df)
```

	Name	Age	City
0	Alice	22	Mumbai
1	Bob	25	Pune
2	Charlie	24	Delhi

Display Specific Columns :-

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 25, 24],
    "City": ["Mumbai", "Pune", "Delhi"]
}

df = pd.DataFrame(data)

print(df["Name"])
print(df["Age"])
```

```
0    Alice
1     Bob
2  Charlie
Name: Name, dtype: object
0     22
1     25
2     24
Name: Age, dtype: int64
```


✓ Reading Data :-

Pandas provides built-in functions to read data from different file formats such as CSV, Excel, JSON, SQL databases, and more.

The most commonly used functions are:

- **read_csv()** – Reads data from a CSV file.
- **read_excel()** – Reads data from an Excel file.

1. read_csv()

Reads data from a CSV (Comma-Separated Values) file and returns a DataFrame.

Syntax :-

```
pd.read_csv("file_name.csv")
```

2. read_excel()

Reads data from an Excel (.xlsx) file and returns a DataFrame.

Syntax :-

```
pd.read_excel("file_name.xlsx")
```

Creating a CSV File :-

```
import pandas as pd

students = {
    "ID": [101, 102, 103, 104, 105,106],
    "Name": ["Alice", "Bob", "Charlie", "David", "Emma","Alex"],
    "Age": [20, 21, 19, 22, 20,24],
    "Course": ["Python", "Java", "C++", "Python", "Java","Ruby"],
    "Marks": [85, 90, 78, 92, 88,95]
}

df = pd.DataFrame(students)

df.to_csv("students.csv", index=False)

print("students.csv created successfully!")

students.csv created successfully!
```

Reading a CSV File :-

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
1	102	Bob	21	Java	90
2	103	Charlie	19	C++	78
3	104	David	22	Python	92
4	105	Emma	20	Java	88
5	106	Alex	24	Ruby	95

Read Only the First 3 Rows :-

```
import pandas as pd

df = pd.read_csv("students.csv",nrows=3)

print(df.head())
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
1	102	Bob	21	Java	90
2	103	Charlie	19	C++	78

✓ Viewing Data :-

After loading a dataset, it is important to inspect its contents. Pandas provides several built-in functions to quickly understand the structure and summary of the data.

The most commonly used functions are:

- **head()** – Displays the first few rows.
- **tail()** – Displays the last few rows.
- **info()** – Displays information about the DataFrame.
- **describe()** – Displays statistical summary of numerical columns.

head() :-

Description: Displays the first 5 rows of the DataFrame by default.

Example :-

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df.head())
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
1	102	Bob	21	Java	90
2	103	Charlie	19	C++	78
3	104	David	22	Python	92
4	105	Emma	20	Java	88

head(n) :-

Description: Displays the first n rows.

Example :-**bold text**

```
print(df.head(3))
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
1	102	Bob	21	Java	90
2	103	Charlie	19	C++	78

tail() :-

Description: Displays the last 5 rows of the DataFrame by default.

Example :-

```
print(df.tail())
```

	ID	Name	Age	Course	Marks
1	102	Bob	21	Java	90
2	103	Charlie	19	C++	78
3	104	David	22	Python	92
4	105	Emma	20	Java	88
5	106	Alex	24	Ruby	95

info() :-

Description: Displays information about the DataFrame, including column names, data types, non-null values, and memory usage.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    ID      6 non-null      int64
1   Name     6 non-null      object
2   Age      6 non-null      int64
3   Course   6 non-null      object
4   Marks    6 non-null      int64
dtypes: int64(3), object(2)
memory usage: 372.0+ bytes
```

describe() :-

Description: Displays statistical information for numerical columns.

```
print(df.describe())
```

	ID	Age	Marks
count	6.000000	6.000000	6.000000
mean	103.500000	21.000000	88.000000
std	1.870829	1.788854	5.966574
min	101.000000	19.000000	78.000000
25%	102.250000	20.000000	85.750000
50%	103.500000	20.500000	89.000000
75%	104.750000	21.750000	91.500000
max	106.000000	24.000000	95.000000

✓ Selecting Data :-

Selecting data is one of the most common operations in Pandas. You can select columns, rows, or specific values using different methods.

The most commonly used methods are:

- Selecting Columns
- Selecting Rows
- `loc[]`
- `iloc[]`

Select a Single Column :-

Description: Selects one column from the DataFrame.

```
print(df["Name"])
```

```
0      Alice
1        Bob
2    Charlie
3      David
4      Emma
5      Alex
Name: Name, dtype: object
```

Select Multiple Columns :-

Description: Selects multiple columns.

```
print(df[["Name", "Marks"]])
```

	Name	Marks
0	Alice	85
1	Bob	90
2	Charlie	78
3	David	92
4	Emma	88
5	Alex	95

Select Rows using loc[]

Description: Selects rows using their index labels.

```
print(df.loc[0])
```

```
print(df.loc[2])
```

```
ID      101
Name    Alice
Age      20
Course  Python
Marks    85
Name: 0, dtype: object
ID      103
Name    Charlie
Age      19
Course   C++
Marks    78
Name: 2, dtype: object
```

Select Rows and Columns using loc[] :-

```
print(df.iloc[0:3, 1:4])
```

	Name	Age	Course
0	Alice	20	Python
1	Bob	21	Java
2	Charlie	19	C++

✓ Data Cleaning :-

Real-world datasets often contain missing values, duplicate records, or inconsistent data. Pandas provides several built-in functions to clean and prepare data before analysis.

The most commonly used functions are:

- `isnull()`
- `notnull()`
- `fillna()`
- `dropna()`
- `duplicated()`
- `drop_duplicates()`

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Alice", "Bob", "Charlie", "David", np.nan],
    "Age": [20, np.nan, 19, 22, 21],
    "Marks": [85, 90, np.nan, 92, 88]
}

df = pd.DataFrame(data)

print(df)
```

	Name	Age	Marks
0	Alice	20.0	85.0
1	Bob	NaN	90.0
2	Charlie	19.0	NaN
3	David	22.0	92.0
4	NaN	21.0	88.0

`isnull()` :-

Description: Checks for missing (NaN) values.

```
print(df.isnull())
```

	Name	Age	Marks
0	False	False	False
1	False	True	False
2	False	False	True
3	False	False	False
4	True	False	False

notnull() :-

Description: Checks for non-missing values.

```
print(df.notnull())
```

	Name	Age	Marks
0	True	True	True
1	True	False	True
2	True	True	False
3	True	True	True
4	False	True	True

fillna() :-

Description: Replaces missing values with a specified value

```
print(df.fillna(0))
```

	Name	Age	Marks
0	Alice	20.0	85.0
1	Bob	0.0	90.0
2	Charlie	19.0	0.0
3	David	22.0	92.0
4	0	21.0	88.0

dropna() :-

Description: Removes rows containing missing values.

```
print(df.dropna())
```

	Name	Age	Marks
0	Alice	20.0	85.0
3	David	22.0	92.0

uplicated() :-

Description: Identifies duplicate rows.

```
print(df.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
dtype: bool
```

✓ Filtering Data :-

Filtering is used to select rows that satisfy a specific condition. It helps you analyze only the data you need.

The most commonly used filtering methods are:

- Comparison Operators (`==`, `!=`, `>`, `<`, `>=`, `<=`)
- Multiple Conditions (`&`, `|`)
- `isin()`

Filter Using `>`

Description: Display students whose marks are greater than 85.

```
print(df[df["Marks"] > 85])
```

	Name	Age	Marks
1	Bob	NaN	90.0
3	David	22.0	92.0
4	NaN	21.0	88.0

Filter Using `==`

Description: Display students enrolled in the Python course.

```
print(df[df["Course"] == "Python"])
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
3	104	David	22	Python	92

Multiple Conditions (&)

Description: Display students whose course is Python and marks are greater than 85.

```
print(df[(df["Course"] == "Python") & (df["Marks"] > 85)])
```

	ID	Name	Age	Course	Marks
3	104	David	22	Python	92

Multiple Conditions (|)

Description: Display students who are in the Python course or have marks greater than 90.

```
print(df[(df["Course"] == "Python") | (df["Marks"] > 90)])
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
3	104	David	22	Python	92
5	106	Alex	24	Ruby	95

Using isin()

Description: Display students enrolled in either Python or Java.

```
print(df[df["Course"].isin(["Python", "Java"])])
```

	ID	Name	Age	Course	Marks
0	101	Alice	20	Python	85
1	102	Bob	21	Java	90
3	104	David	22	Python	92
4	105	Emma	20	Java	88

✓ GroupBy :-

The `groupby()` function is used to group rows that have the same value in one or more columns. After grouping, you can perform operations like calculating the sum, average, count, and more.

It is widely used for data analysis and reporting.

Syntax

```
df.groupby("column_name")
```

Group by Course

Description: Groups students based on the Course column.

```
group = df.groupby("Course")
```

```
print(group)
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x79ba271a3a10>
```

Calculate Average Marks :-

Description: Calculates the average marks for each course.

```
print(df.groupby("Course")["Marks"].mean())
```

```
Course
C++      78.0
Java     89.0
Python   88.5
Ruby     95.0
Name: Marks, dtype: float64
```

Calculate Total Marks

Description: Calculates the total marks for each course.

```
print(df.groupby("Course")["Marks"].sum())
```

```
Course
C++      78
Java    178
Python  177
Ruby     95
Name: Marks, dtype: int64
```

Count Students

Description: Counts the number of students in each course.

```
print(df.groupby("Course")["Name"].count())
```

```
Course
C++      1
Java     2
Python   2
Ruby     1
Name: Name, dtype: int64
```

✓ Merge & Concatenate :-

Pandas provides functions to combine multiple DataFrames.

- `merge()` – Combines DataFrames using a common column.
- `concat()` – Stacks DataFrames row-wise or column-wise.

```
import pandas as pd

students = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Alice", "Bob", "Charlie"]
})

marks = pd.DataFrame({
    "ID": [101, 102, 103],
    "Marks": [85, 90, 78]
})

print(pd.merge(students, marks, on="ID"))
```

	ID	Name	Marks
0	101	Alice	85
1	102	Bob	90
2	103	Charlie	78

`concat()` :-

```
df1 = pd.DataFrame({"Name": ["Alice", "Bob"]})
df2 = pd.DataFrame({"Name": ["Charlie", "David"]})

print(pd.concat([df1, df2], ignore_index=True))
```

	Name
0	Alice
1	Bob
2	Charlie
3	David

✓ Statistical Functions :-

Pandas provides built-in functions for statistical analysis.

- `sum()`
- `mean()`

- `min()`
- `max()`
- `count()`
- `std()`

```
print(df["Marks"].sum())
print(df["Marks"].mean())
print(df["Marks"].min())
print(df["Marks"].max())
print(df["Marks"].count())
print(df["Marks"].std())
```

```
528
88.0
78
95
6
5.966573556070519
```

✓ Date & Time :-

The `to_datetime()` function converts dates into Pandas datetime format, allowing you to extract components like year, month, and day.

```
import pandas as pd

dates = pd.DataFrame({
    "Date": ["2026-01-15", "2026-02-20", "2026-03-10"]
})

dates["Date"] = pd.to_datetime(dates["Date"])

print(dates)

print(dates["Date"].dt.year)
print(dates["Date"].dt.month)
print(dates["Date"].dt.day)
```

```
      Date
0 2026-01-15
1 2026-02-20
2 2026-03-10
0      2026
1      2026
2      2026
Name: Date, dtype: int32
0      1
```

```
1    2
2    3
Name: Date, dtype: int32
0    15
1    20
2    10
Name: Date, dtype: int32
```

✓ Exporting Data :-

Pandas allows you to save DataFrames to different file formats.

- `to_csv()`
- `to_excel()`

```
df.to_csv("output.csv", index=False)
```

```
df.to_excel("output.xlsx", index=False)
```

Conclusion :-

Pandas is a super helpful Python tool that acts like a powerful, automated version of Excel. It helps you open, read, and organize large amounts of messy data into neat tables called DataFrames. With Pandas, you can easily clean up missing information, filter out what you do not need, and sort your data in just a few seconds.

Ultimately, Pandas saves time by doing the heavy lifting of data preparation. It easily connects with other Python tools used for making charts or building smart AI models. It is the essential first step for anyone wanting to turn raw, confusing numbers into clear,